

Get ready to build production-minded AI.

From a clean machine to one verified AI reply — safely and repeatably.

AGENTIC AI · PRODUCTION

Today we install the tools, protect the API key, isolate dependencies, and make a real model call.

- Python
- VS Code
- .venv
- .env
- First API call



What ‘done’ looks like today.

Five pieces of evidence — not a vague feeling that setup probably worked.

CHECK 01

Python runs

The terminal reports Python 3.11 or newer.

CHECK 02

Editor opens

VS Code opens the course repository.

CHECK 03

Environment is isolated

The terminal shows the active .venv.

CHECK 04

Secret stays local

.env exists locally and Git ignores it.

CHECK 05

AI replies

The first-call script returns text and usage information.

CHECK 06

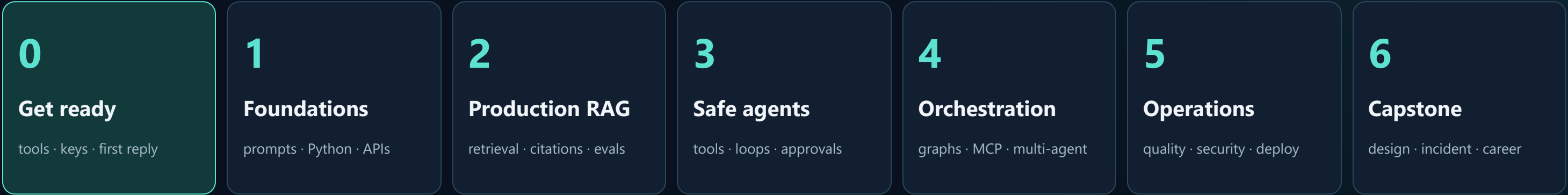
You can explain it

You know what crossed the network and what may cost money.

Day 0 evidence: the setup checker passes, `.env` is ignored, and the first AI call succeeds.

Where Day 0 fits in the 40-day journey.

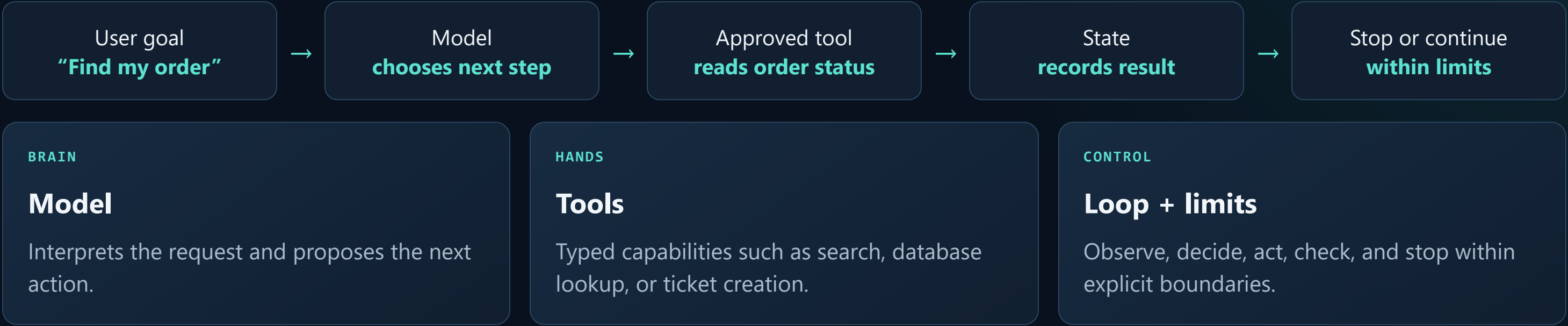
Each phase adds capability and the evidence needed to operate it responsibly.



Course promise: design, build, evaluate, secure, deploy, observe, operate, and explain an agent system.

What is Agentic AI?

A model that can pursue a goal through a bounded loop using approved tools and state.



Model call, chatbot, workflow, or agent?

Use the least complex design that solves the problem.

DESIGN	BEST FIT	EXAMPLE
Single model call	One transformation	Summarize a review
Chatbot	Conversation without autonomous actions	Explain a policy
Deterministic workflow	Known, fixed steps	Validate → transform → save
Agent	Next step depends on observations	Investigate a support case with tools

Production habit: agentic behavior must earn its complexity.

The local toolkit — in plain language.

Each item has one job in the development loop.

RUNTIME

Python

Runs the course programs and SDKs.

EDITOR

VS Code

Edits files and provides an integrated terminal.

HISTORY

Git

Tracks changes and helps prove how a project evolved.

ACCESS

Provider account

Issues the API key and records model usage.

BOUNDARY

Virtual environment

Keeps this project's packages isolated.

WORKSPACE

Course repository

Contains lessons, helpers, labs, and checks.

The terminal is where we prove things.

A command, observable output, and a clear pass condition.

TERMINAL

Copied

```
python --version
```

EXPECTED

Python 3.11 or newer

If Windows cannot find Python, PATH is usually the issue. On macOS, try `python3`.

Opening an installer is an action. Seeing the expected version is evidence.

Install Python and VS Code.

Show the finished result first, then install, then verify again.

1

Install Python 3.11+

Windows: select "Add python.exe to PATH" on the first installer screen.

2

Open a new terminal

Old terminals may not see a newly updated PATH.

3

Run the version check

Use `python3 --version` on macOS if needed.

4

Install and open VS Code

Open the course repository folder, not a single loose file.

Pause point: continue only when the version command works and VS Code opens the repository.

Read the error before changing anything.

One controlled troubleshooting loop.

ERROR

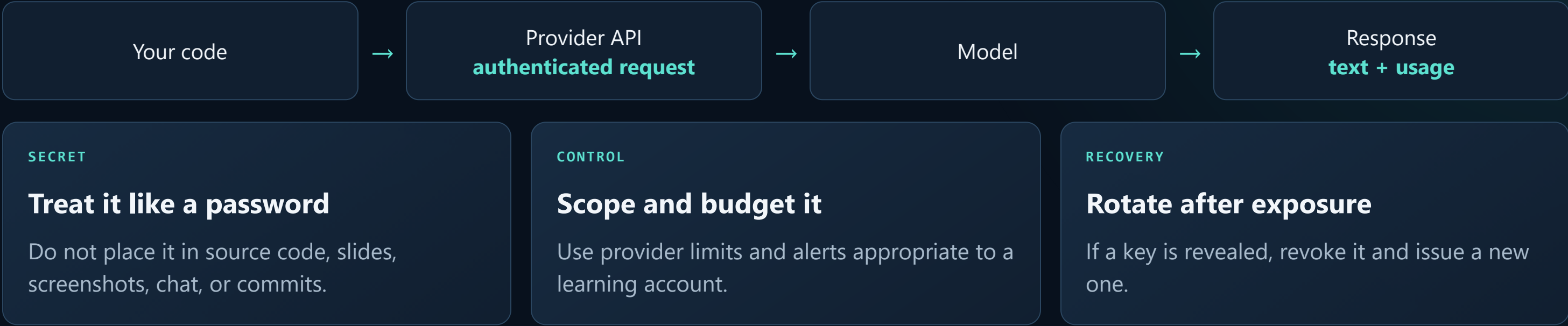
```
python : The term 'python' is not recognized...
```

1. **Read** the first useful error line.
2. **Classify** it: command discovery, package, secret, network, or code?
3. **Change one thing.**
4. **Run the same check again.**

Likely Windows fix: reinstall Python with “Add python.exe to PATH”, then open a new terminal.

An API key is a credential.

It identifies and authorizes requests to a paid service.



The model call crosses a network boundary. Prompt content is sent to the selected provider. Do not use sensitive data in this first exercise.

Template in Git. Secret on your machine.

Two files with different jobs.

FILES

Copy

```
# .env.example – safe template
LLM_PROVIDER=anthropic
ANTHROPIC_API_KEY=replace-me

# .env – local secret
ANTHROPIC_API_KEY=your-real-key
```

GIT IGNORE

Copy

```
# .gitignore
.env
.env.local
*.key
*.pem
```

`.env.example` documents the required names. `.env` contains real values and must remain untracked.

Verify that Git ignores the secret.

Safety checks should be observable too.

TERMINAL · FROM REPOSITORY ROOT

Copy

```
git check-ignore -v .env
```

EXPECTED SHAPE

```
.gitignore:2:.env      .env
```

No output? Stop before committing. Confirm the file is named exactly `.env` and the ignore rule exists.

Create the project's private toolbox.

A virtual environment isolates Python packages for this repository.

TERMINAL

Copy

```
python -m venv .venv
.venv\Scripts\Activate.ps1

# macOS / Linux:
# source .venv/bin/activate
```

EXPECTED

(.venv) appears in the prompt

Activation changes which Python and pip this terminal uses.

If PowerShell blocks activation, use the exact troubleshooting guidance in the lab README; do not weaken system-wide policy casually.

Install pinned dependencies and the shared helper.

The repository defines the known-good package versions.

TERMINAL

Copy

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
```

DEPENDENCIES

requirements.txt

Pins shared provider SDKs and course packages.

EDITABLE INSTALL

pip install -e .

Makes shared.Ilm importable from any lab folder.

CONSISTENCY

python -m pip

Ties pip to the active Python interpreter.

Create local configuration safely.

Copy the template, choose one provider, and paste the key off-camera.

TERMINAL

Copy

```
copy .env.example .env

# macOS / Linux:
# cp .env.example .env
```

- ☐ Provider selected
- ☐ Real key added to .env
- ☐ No quotes or extra spaces
- ☐ git check-ignore confirms .env

Recording rule: stop screen capture before pasting a real key. Resume only after the file is closed.

Run the setup gate.

The checker validates the foundation without making a paid model call.

TERMINAL

Copy

```
python phase-0-get-ready/day-00/labs/lab-0.5b-setup-project/solution/check_setup.py
```

ILLUSTRATIVE OUTPUT

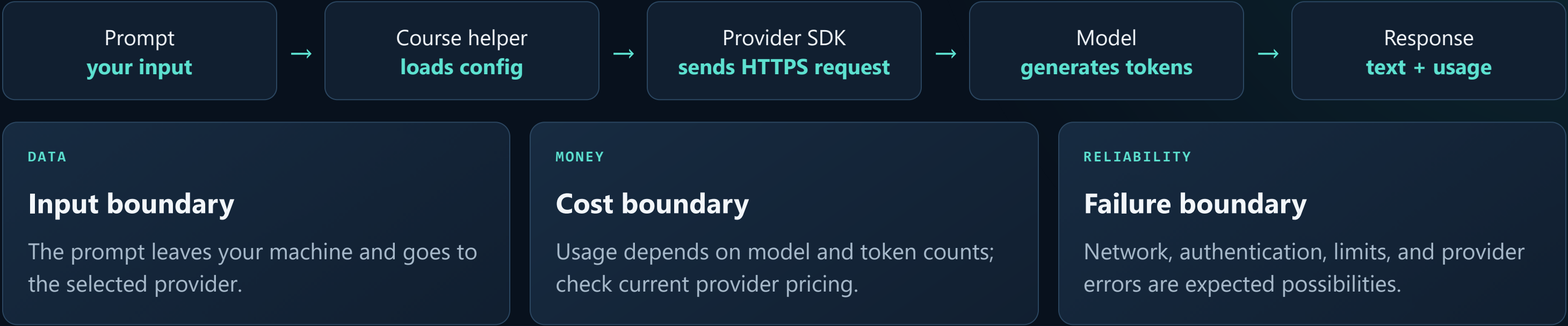
```
OK Python version
OK package: anthropic
OK package: openai
OK package: dotenv
OK ANTHROPIC_API_KEY is set
```

```
All checks passed.
```

This check confirms presence and imports. It does not prove the key is valid, and it does not spend model tokens.

What happens during one AI request?

Understand the boundary before running the code.



The smallest useful program.

Import → describe intent → call → display evidence.

PYTHON

Copy

```
from shared.llm import ask

prompt = "Welcome a beginner to an Agentic AI course in one sentence."
result = ask(prompt)
print(result)
```

The helper keeps provider wiring out of the beginner lab. Later lessons expose requests, schemas, failures, and controls in more detail.

Run the first real AI call.

This is the first step today that contacts a model provider.

TERMINAL

Copy

```
python phase-0-get-ready/day-00/labs/lab-0.5c-hello-ai/solution/hello_ai.py
```

EXPECTED SHAPE — WORDING WILL VARY

```
AI: Welcome! You are about to learn how to build useful AI systems...
```

```
provider=... model=... input_tokens=... output_tokens=...
```

A model response is variable. Verify the response shape and metadata, not an exact sentence.

Change one variable and observe.

A small experiment turns a copied example into understanding.

BEFORE

Welcome a beginner to an Agentic AI course in one sentence.

YOUR VARIATION

Explain an API key to a ten-year-old in two sentences.

Predict what will change. Run once. Compare the response and token usage.

Find the failing layer.

The same symptom can come from different boundaries.

SYMPTOM	LIKELY LAYER	FIRST CHECK
python not recognized	Runtime / PATH	Version command in a new terminal
No module named shared	Project install	Active .venv and <code>pip install -e .</code>
Missing API key	Configuration	Provider name and local .env variable
401 / authentication	Provider access	Correct key, active account, accidental whitespace
429 / rate limit	Provider capacity or account limit	Retry guidance, limits, budget, request rate
Timeout / connection	Network or provider	Connectivity, proxy, status page, then retry

Know what you send and what you spend.

Day 0 establishes the questions; later phases add measurement and budgets.

CHECK

Before the call
Is this prompt safe to send? Which provider and model are selected?

OBSERVE

After the call
What usage was reported? Was the response necessary and useful?

CONTROL

At the account
Are budget limits, alerts, and access controls configured?

Approximate request cost

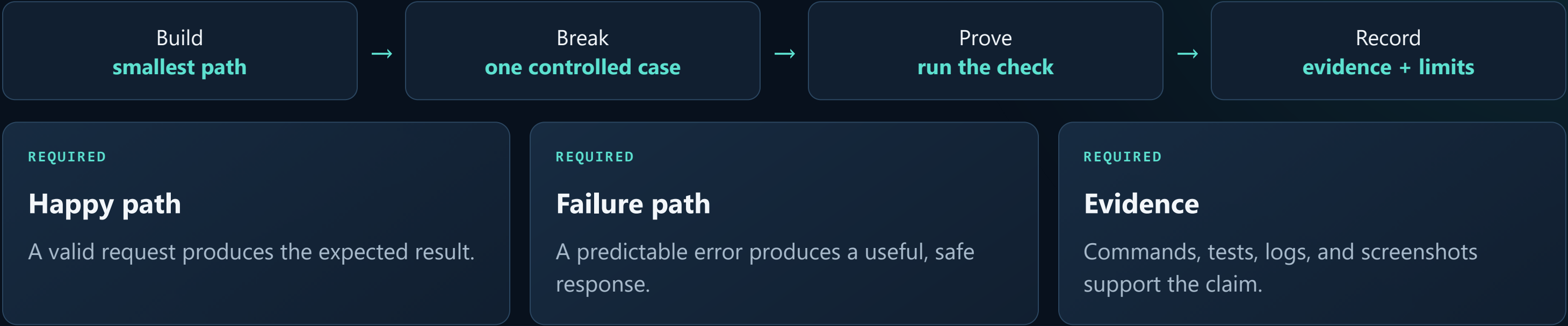
$$\text{input tokens} \times \text{input rate} + \text{output tokens} \times \text{output rate}$$

Use current provider pricing and units. Cached or special token types may be priced differently.

Never claim “free” or a fixed cost without checking the active model, account, and current provider pricing.

Build → break → prove → record.

A repeatable loop for every lab and production change.



Can you explain the setup boundary?

Pause, answer aloud, then reveal each answer.

▼ 1. Why use a virtual environment?

It isolates this project's interpreter context and packages, making setup more reproducible and reducing conflicts with other projects.

▶ 2. Why commit `.env.example` but never `.env`?

▶ 3. Does a passing setup checker prove the API key is valid?

▶ 4. What crosses the network in the first call?

Day 0 readiness check.

Mark every item using evidence from your machine.

☒ Python 3.11+ prints in a new terminal

☒ The .venv is active

☒ .env contains the selected provider key

☒ The setup checker passes

☒ No secret appears in code, Git, screenshots, or terminal history

☒ VS Code opens the repository folder

☒ Pinned packages and editable helper are installed

☒ git check-ignore confirms .env is ignored

☒ The first AI call returns text and usage

☒ I can explain the data and cost boundary

10 / 10 verified

Day 0 release gate passed.

Your machine can now make a controlled AI request.

The foundation is local, safe, reproducible, and verified.

READY

Local

Python, VS Code, and the repository work on your machine.

READY

Safe

The real key stays in an ignored local file.

READY

Verified

Checks prove setup, and a real response proves the request path.

Next: Day 1 — Meet the LLM. Tokens, context, prompt anatomy, and a repeatable first-call script.

Day 0 recording companion.

Files, sources, and delivery notes for the instructor.

SCRIPT

Read-aloud transcript

speaker-notes.md contains the exact narration and action cues.

RUNBOOK

Recording runbook

README.md covers preparation, safe demo rules, and lesson segments.

CURRICULUM

Source alignment

The deck follows PRODUCTION-READY-COURSE-TOC.md and the existing Day 0 labs.

[Production course TOC](#)

[Existing Day 0 source](#)

All terminal outputs in the slides are labeled illustrative. During a live recording, describe the output you actually observe.